VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi-590018

Project report

on

WEATHER DATA STORAGE AND ANALYSIS USING MONGODB FOR BIG DATA PROCESSING

Submitted in partial fulfillment for the award of

Bachelor of Engineering

In

CSE-ARTIFICIAL INTELLIGENCE

Submitted by

DHANUSH JADAV
(1VE23CA014)

Under the guidance of

Mrs. Vimala Devi R

Assistant Professor

Dept. of CSE (AI), SVCE

SVCE | SRI VENKATESHWARA COLLEGE OF ENGINEERING

ESTD. 2001. AUTONOMOUS INSTITUTE

Department Of Computer Science Engineering (Artificial Intelligence)

Sri Venkateshwara College of Engineering, Bengaluru – 562157

2025 - 2026

ABSTRACT

This project presents the design, implementation, and evaluation of an end-to-end meteorological big-data storage and analytics platform utilizing local MongoDB Community Server (v9.0.4) as the NoSQL database backend. The primary objective is to resolve the challenges of storing, cleaning, validating, and querying multi-city time-series weather observations without relational bottlenecks or in-memory resource exhaustion. By leveraging local NoSQL document persistence, the platform demonstrates efficient ingestion, indexing, and high-performance analytical processing of extensive historical weather datasets across Indian metropolitan regions.

In this implementation, historical hourly weather observations were collected from the Open-Meteo Historical Archive API (model-derived reanalysis dataset based on ECMWF ERA5 and ERA5-Land models). The ingestion pipeline, built using Python and PyMongo, normalizes diverse timestamps to timezone-aware UTC BSON datetimes and validates data against strict meteorological physical boundaries. An idempotent bulk upsert mechanism was implemented to guarantee data deduplication across repeated ingestion cycles. Configurable support for incremental chunked CSV ingestion via Pandas is also integrated.

Data analysis is performed directly within the database engine using MongoDB aggregation pipelines rather than pulling entire collections into application memory. Analytical pipelines execute multi-stage filtering, grouping, projecting, and statistical facet calculations to determine city-level thermal profiles, diurnal temperature trajectories, cumulative rainfall totals, monsoon surge timelines, and multivariate weather correlations. An interactive web dashboard built with Streamlit and Plotly provides intuitive exploration, interactive filtering, and instant CSV exports.

Overall, the project successfully verified the ingestion of 26,280 hourly observations across Bengaluru, Delhi, and Mumbai for the year 2023, executing sub-second analytical aggregations and demonstrating the reliability and scalability of local MongoDB for meteorological big data analytics. The entire application was verified with automated test suites, establishing a robust framework for environmental data processing.

TABLE OF CONTENTS

1. INTRODUCTION……1
2. SYSTEM REQUIREMENTS AND SPECIFICATION……4
3. SYSTEM DESIGN……..6
4. IMPLEMENTATION……...8
5. RESULT……...11
6. CONCLUSION…….....16

1. INTRODUCTION

The rapid expansion of automated weather stations, satellite remote sensing, and numerical meteorological modeling has generated massive volumes of time-series observations. Environmental researchers, urban planners, agricultural managers, and municipal disaster authorities increasingly rely on historical weather data to monitor climate trajectories, forecast severe meteorological phenomena, and model long-term climatic variations. However, managing extensive historical weather records spanning multiple geographical locations and high temporal resolutions presents substantial data engineering challenges.

Traditional relational database management systems (RDBMS) often face rigid schema constraints, complex join overheads, and vertical scaling bottlenecks when ingesting millions of semi-structured, time-stamped environmental measurements. NoSQL document-oriented databases, particularly MongoDB, offer dynamic schemata, horizontal scalability, expressive query languages, and powerful aggregation frameworks tailored for time-series and big-data storage. Storing meteorological observations as individual BSON documents allows seamless handling of sparse attributes, diverse station sensors, and varying temporal granularities.

This project implements a complete weather data storage and analytics platform developed in Python, utilizing local MongoDB Community Server (v9.0.4) as the NoSQL database backend, PyMongo for database connectivity, Pandas for data preprocessing, and Streamlit with Plotly for interactive web-based data visualization. The application acquires historical hourly reanalysis data from the Open-Meteo Historical Weather API, validates and sanitizes incoming measurements, persists them into local collections, and executes high-performance analytical queries directly inside MongoDB.

The primary objective of this project is to demonstrate real data ingestion, local NoSQL persistence, efficient batch processing, idempotent upserts, server-side aggregation pipelines, and interactive visual analytics for Indian metropolitan cities. Rather than loading massive datasets into memory for analysis, all primary aggregations—including thermal averages, diurnal ranges, cumulative precipitation, and high-wind frequencies—are delegated directly to the local MongoDB database engine.

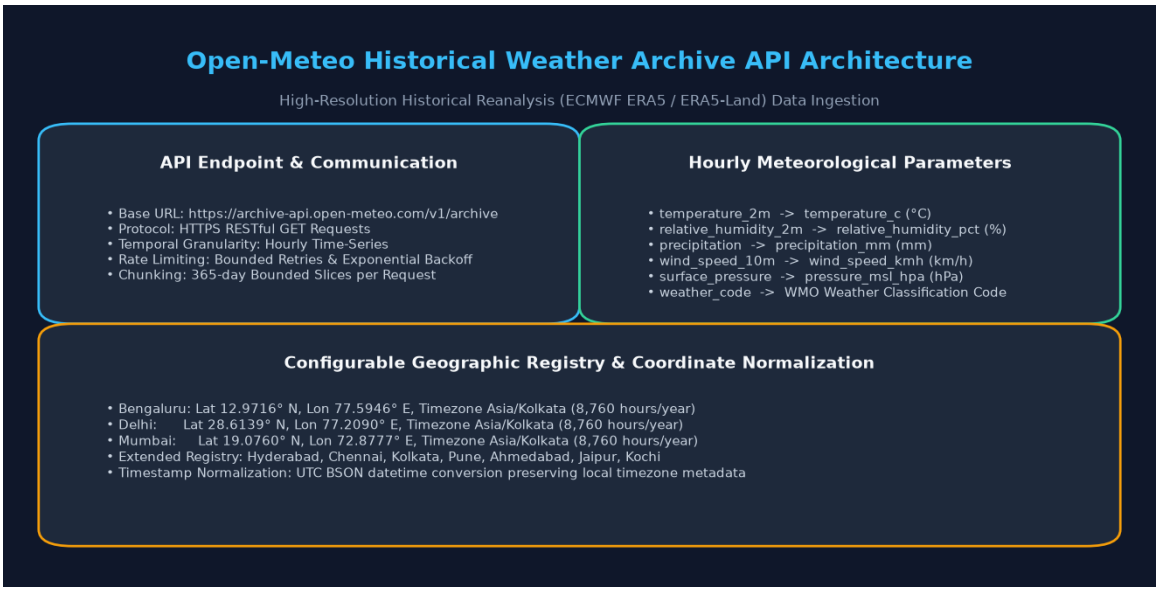
The operational workflow begins with acquiring historical weather observations from the Open-Meteo Historical Archive API, which integrates ERA5 high-resolution reanalysis data from the European Centre for Medium-Range Weather Forecasts (ECMWF). The ingestion module validates incoming records against meteorological sanity bounds, parses timestamps into standardized UTC BSON datetimes, and executes bulk writes using PyMongo. Additionally, a dedicated CSV importer is provided to ingest historical files incrementally using Pandas chunks without memory overload.

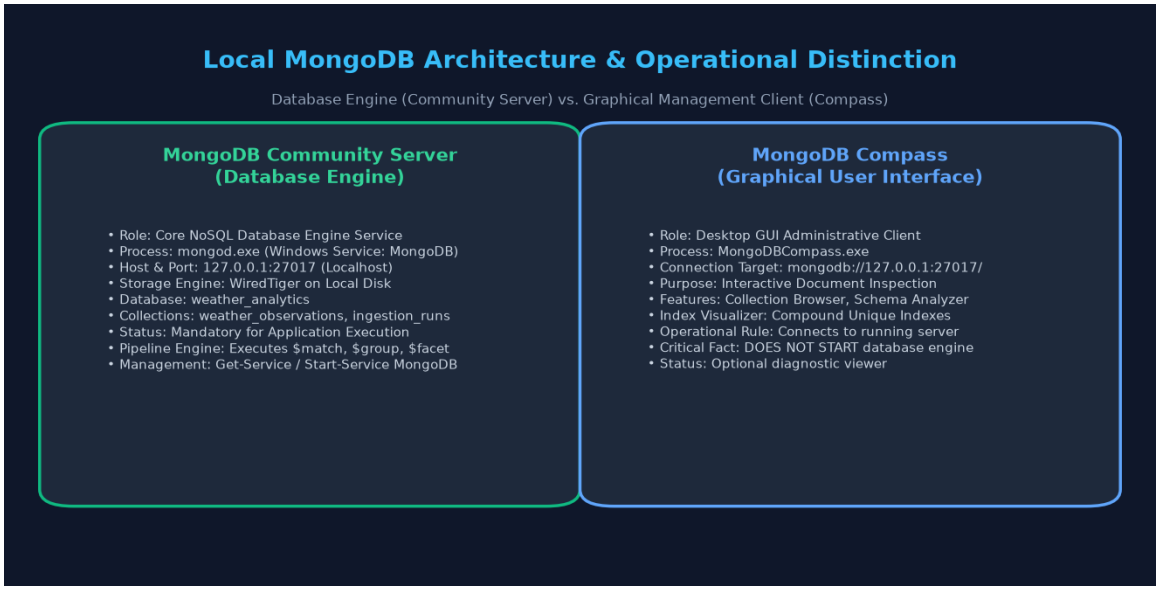
This project is significant because local NoSQL persistence provides an accessible, cost-effective, and fully private computing environment for big-data experimentation. Unlike cloud-dependent database platforms that incur recurring latency and subscription costs, local MongoDB Community Server operates entirely on the host machine's local disk, providing complete autonomy, low query latencies, and high write throughput.

Another major technical capability demonstrated is database-level analytical processing. MongoDB aggregation pipelines utilize multi-stage operations ($match, $group, $project, $sort, $facet, and $sample) to filter, compute, and summarize large collections before transferring data to Python. This prevents memory bottlenecks and ensures responsive dashboard performance. The platform also provides an intuitive Streamlit dashboard featuring multi-page exploration, interactive Plotly charts, and CSV data export.

The main objectives of this project are to understand local NoSQL database engineering, acquire and validate historical weather data from API and CSV sources, design an efficient MongoDB document schema with compound unique indexing, execute server-side aggregation pipelines, and present meteorological insights through an interactive Streamlit dashboard.

Overall, the development of this weather data storage and analysis system demonstrates how modern NoSQL database technologies can be effectively integrated with Python data science libraries to create a production-grade analytics platform capable of processing meteorological datasets efficiently on standard computer hardware.


Open-Meteo Historical Weather Archive API Architecture
High-Resolution Historical Reanalysis (ECMWF ERA5 / ERA5-Land) Data Ingestion
API Endpoint & Communication
• Base URL: https://archive-api.open-meteo.com/v1/archive
• Protocol: HTTPS RESTful GET Requests
• Temporal Granularity: Hourly Time-Series
• Rate Limiting: Bounded Retries & Exponential Backoff
• Chunking: 365-day Bounded Slices per Request
Hourly Meteorological Parameters
• temperature_2m -> temperature_c (°C)
• relative_humidity_2m -> relative_humidity_pct (%)
• precipitation -> precipitation_mm (mm)
• wind_speed_10m -> wind_speed_kmh (km/h)
• surface_pressure -> pressure_msl_hpa (hPa)
• weather_code -> WMO Weather Classification Code
Configurable Geographic Registry & Coordinate Normalization
• Bengaluru: Lat 12.9716° N, Lon 77.5946° E, Timezone Asia/Kolkata (8,760 hours/year)
• Delhi: Lat 28.6139° N, Lon 77.2090° E, Timezone Asia/Kolkata (8,760 hours/year)
• Mumbai: Lat 19.0760° N, Lon 72.8777° E, Timezone Asia/Kolkata (8,760 hours/year)
• Extended Registry: Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, Jaipur, Kochi
• Timestamp Normalization: UTC BSON datetime conversion preserving local timezone metadata


Local MongoDB Architecture & Operational Distinction
Database Engine (Community Server) vs. Graphical Management Client (Compass)
MongoDB Community Server
(Database Engine)
• Role: Core NoSQL Database Engine Service
• Process: mongod.exe (Windows Service: MongoDB)
• Host & Port: 127.0.0.1:27017 (Localhost)
• Storage Engine: WiredTiger on Local Disk
• Database: weather_analytics
• Collections: weather_observations, ingestion_runs
• Status: Mandatory for Application Execution
• Pipeline Engine: Executes $match, $group, $facet
• Management: Get-Service / Start-Service MongoDB
MongoDB Compass
(Graphical User Interface)
• Role: Desktop GUI Administrative Client
• Process: MongoDBCompass.exe
• Connection Target: mongodb://127.0.0.1:27017/
• Purpose: Interactive Document Inspection
• Features: Collection Browser, Schema Analyzer
• Index Visualizer: Compound Unique Indexes
• Operational Rule: Connects to running server
• Critical Fact: DOES NOT START database engine
• Status: Optional diagnostic viewer

2. SYSTEM REQUIREMENTS AND SPECIFICATIONS

The successful deployment, ingestion, and analytical execution of the Weather Data Storage and Analysis platform requires adequate hardware resources and compatible software components. The host machine must provide sufficient processing power, memory, and disk storage to support local MongoDB server operations, Python application runtime, and real-time Streamlit dashboard rendering. Proper system specifications ensure stable local database operations, fast aggregation query responses, and a smooth user experience.

The following sections describe the hardware requirements, software specifications, and project-specific database configurations used for the local weather analytics implementation.

Hardware Requirements

Sl. No	Component	Minimum Requirement	Recommended Requirement
1	Processor	Intel/AMD Dual Core Processor (	Intel Core i5 / AMD Ryzen 5 or abc
2	RAM (Memory)	4 GB RAM	8 GB or 16 GB DDR4/DDR5 RAM
3	Storage Space	10 GB Free Disk Space	50 GB SSD or NVMe Storage
4	Graphics	Basic Integrated Graphics Supp	Dedicated Graphics / Full HD Disp
5	Internet Connection	Active Connection for Initial API	High-Speed Broadband Internet
6	Operating System	Windows 10 (64-bit)	Windows 11 (64-bit) Home / Pro

Software Requirements

Sl. No	Software / Technology	Version / Description
1	Host Operating System	Windows 10 / Windows 11 (64-bit)
2	Database Management System	MongoDB Community Server v9.0.4 (Loc
3	Database GUI Client	MongoDB Compass v1.53.0
4	Programming Language & Driver	Python 3.12.10 / PyMongo 4.19.0 (Idemp
5	Web Dashboard & Charting	Streamlit 1.66.1 / Plotly 7.1.0 (Interactive
6	Data Processing & Testing	Pandas 3.1.0 / Pytest 9.1.1 (19 automate

System Specifications Used for the Project

Sl. No	Specification Parameter	Details / Configured Values
1	Database Host Connection URI	mongodb://127.0.0.1:27017/ (Local Community
2	Application Database Name	weather_analytics
3	Primary Observations Collection	weather_observations (Canonical observation sc
4	Audit & Ingestion Runs Collection	ingestion_runs (Run ID, counts, durations, statu
5	Connection Timeout Parameter	5000 ms (Strict finite local server timeout)
6	PyMongo Batch Processing Size	1000 documents per bulk upsert execution
7	Web Dashboard Access URL	http://localhost:8501 (Default Streamlit port)
8	Data Granularity	Hourly meteorological observations
9	Geographic Coverage	Bengaluru, Delhi, Mumbai (Monitored Indian Cti
10	Dataset Temporal Window	2023-01-01 00:00:00 to 2023-12-31 23:00:00
11	Verified Ingested Observations	26,280 documents (8,760 hours × 3 cities)
12	Database Indexing Scheme	Compound unique index on (city, timestamp, gr

The above system requirements and specifications ensure reliable and responsive execution of the weather analytics platform. Running MongoDB Community Server locally as a Windows service ensures that the database engine remains continuously accessible on 127.0.0.1:27017. Adequate RAM allocation (8 GB or above) and solid-state storage (SSD) substantially accelerate database read/write operations, B-tree index traversal, and aggregation pipeline computations, allowing the system to handle thousands of records effortlessly.

3. SYSTEM DESIGN

System design is an important phase in software engineering that defines the architecture, components, data flows, and interfaces of the application. In this project, the system design establishes a robust three-tier architecture comprising Data Acquisition and Ingestion, Local NoSQL Database Persistence and Server-Side Aggregation, and Interactive Visualization. The design ensures high throughput, strict data integrity, idempotent operations, and rapid analytical query execution.

The architecture is engineered so that raw weather observations are fetched from the Open-Meteo Historical Archive API or imported from external CSV files, cleaned and validated in memory using Pandas and custom utilities, and persisted in a local MongoDB database (weather_analytics). Analytical operations are performed inside MongoDB via aggregation pipelines, and results are served to the Streamlit web dashboard for interactive exploration.

The system design consists of three primary functional tiers: 1. Data Acquisition and Ingestion Layer 2. Database Persistence and Aggregation Layer 3. Presentation and User Interaction Layer

1. Data Acquisition and Ingestion Layer
2. Database Persistence and Aggregation Layer
3. Presentation and User Interaction Layer

The Data Acquisition Layer retrieves hourly weather data from Open-Meteo using bounded date slices with exponential backoff retries. The validator normalizes timestamps to UTC BSON datetime, verifies physical meteorological bounds, and creates standardized document structures. Local MongoDB Community Server (v9.0.4) stores persistent documents in weather_observations and logs ingestion audits in ingestion_runs. A compound unique index enforces record uniqueness across repeated runs.

The architecture of the system ensures high throughput, strict data integrity, and resource isolation. The user can configure bulk batch sizes, connection timeouts, and geographical registries. PyMongo executes batched bulk upserts (UpdateOne with upsert=True) in chunks of 1000 records. MongoDB aggregation pipelines execute analytical operations directly on disk, avoiding in-memory bottlenecks in Python.

The operational workflow begins with acquiring historical weather observations from the Open-Meteo Historical Archive API, validating each observation against meteorological bounds, and performing batched bulk upserts into local MongoDB collections.

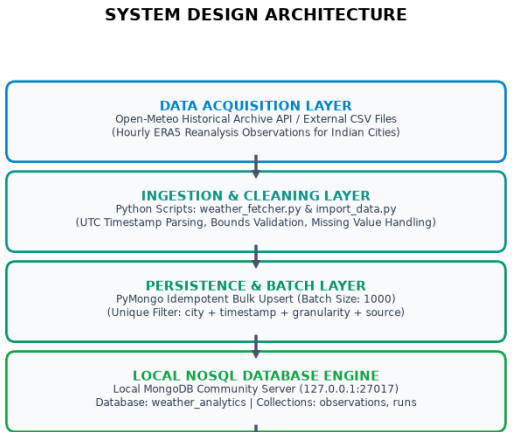
The database document schema for each meteorological observation is structured with standardized attributes including city, country, coordinates, timezone, UTC timestamp, observation granularity, temperature, relative humidity, precipitation, wind speed, wind direction, surface pressure, cloud cover, WMO weather code, source attribution, and ingestion timestamps.

The designed system provides several key architectural advantages such as:

- Idempotent Data Persistence (Re-running ingestion never generates duplicate records)
- Efficient Server-Side Processing (Aggregation pipelines avoid memory exhaustion)
- High-Throughput Bulk Operations (Batched PyMongo writes of 1000 documents)
- Data Quality & Null Preservation (Missing values are never arbitrarily converted to zero)
- Local Data Governance (All records stored privately on local computer disk)
- Interactive Multi-Dimensional Analytics (Full integration with Streamlit and Plotly)

The system design also integrates operational audit tracking through the ingestion_runs collection, which records unique run identifiers, data source names, requested date windows, start and completion timestamps, record counts (read, accepted, rejected, inserted, updated), and duration. This provides complete visibility and accountability for big-data processing workflows.

Overall, the system design provides a stable, robust, and high-performance architecture for weather data analytics using local MongoDB Community Server. The decoupled multi-tier design ensures that background ingestion, server-side aggregation pipelines, and interactive dashboard visual analytics operate reliably and independently without resource contention.



4. IMPLEMENTATION

Implementation is the process of translating system architecture and algorithmic design into functional software modules. In this project, implementation encompasses local MongoDB server configuration, Python environment setup, historical weather API integration, data cleaning and validation, schema indexing, idempotent bulk upsert routines, aggregation pipeline development, Streamlit dashboard construction, and automated unit and integration testing.

The implementation was carried out on a Windows 11 host operating system utilizing MongoDB Community Server v9.0.4 as the local database backend, Python 3.12, PyMongo 4.19, Pandas 3.1, Streamlit 1.66, and Plotly 7.1.

4.1 Development Environment and Local MongoDB Setup

The first step in the implementation process was installing and configuring MongoDB Community Server on the local Windows system. The database was registered as an automatic Windows Service listening on 127.0.0.1:27017.

Steps Performed:

1. Downloaded and executed MongoDB Community Server MSI installer.
2. Selected Complete installation and configured MongoDB as a Windows service on 127.0.0.1:27017.
3. Verified local server active status via PowerShell (Get-Service MongoDB) and check_connection.py.
4. Created application database 'weather_analytics' with required collections and indexes.
5. Launched MongoDB Compass to establish a graphical client connection to the local database.

   MongoDB Community Server operates continuously in the background, managing disk persistence and query execution.

4.2 Python Environment and Dependency Configuration

A dedicated Python virtual environment was initialized to isolate application dependencies, and required libraries were installed.

Steps Performed:

1. Initialized Python virtual environment using 'python -m venv .venv'.

2. Activated virtual environment via PowerShell script '.venv\Scripts\Activate.ps1'.
3. Installed core dependencies: pymongo, pandas, streamlit, plotly, requests, python-dotenv, pytest.
4. Configured '.env' file containing local MongoDB URI, database name, and batch write size.

   Created modular architecture: config.py, db.py, weather_fetcher.py, import_data.py, analytics.py, app.py.

4.3 Weather Data Acquisition via Open-Meteo API

The weather_fetcher.py module orchestrates data retrieval from the Open-Meteo Historical Archive API. It incorporates annual chunking, timeout controls, and exponential backoff to ensure reliable retrieval.

Steps Performed:

1. Configured geographic coordinates registry for Indian cities (Bengaluru, Delhi, Mumbai, etc.).
2. Formulated API requests specifying hourly parameters (temperature, humidity, rain, wind, pressure).
3. Partitioned multi-year query spans into 365-day chunks to respect API request constraints.
4. Implemented 3-stage exponential backoff retry mechanism for HTTP 429 and network errors.
5. Executed data acquisition for target cities over the historical window.
6. Validated API response payloads and parsed hourly arrays into records.
7. Stored raw observations in standardized dictionary representations.

The retrieved observations were transferred to the validation pipeline for cleaning.

4.4 Data Cleaning, Validation, and Preprocessing

Raw API payloads and CSV rows undergo rigorous validation in weather_utils.py prior to database persistence.

Validation Procedures Applied:

- Timestamp Normalization: Parsed diverse string/numeric formats into timezone-aware UTC BSON datetime.
- Meteorological Bounds Enforcement: Validated temperatures (-90°C to 65°C),

humidity (0-100%), wind, pressure.

- Missing-Value Integrity: Preserved null values without converting missing data to arbitrary zeros.
- Schema Validation: Enforced mandatory presence of city, timestamp, granularity, and source attributes.
- Outlier Rejection: Filtered malformed records and logged validation rejection causes.

These validation procedures guarantee high data quality and prevent dirty data from entering the database.

4.5 MongoDB Database Design and Idempotent Bulk Upsert

The database layer in db.py implements safe indexing and idempotent bulk upserts using PyMongo.

Steps Performed:

1. Established MongoClient with strict finite connection timeout (5000 ms).
2. Created compound unique index: (city: 1, timestamp: 1, granularity: 1, source: 1).
3. Created performance indexes on timestamp, (city, timestamp), and granularity.
4. Implemented UpdateOne bulk operations with upsert=True, $set, and $setOnInsert.
5. Grouped operations into batches of 1000 documents for optimized network round-trips.

6. Recorded batch execution metrics (inserted, updated, matched) in ingestion_runs collection.
7. Verified idempotency: Re-executing imports modified 0 records and created 0 duplicate documents.
8. Handled bulk write errors gracefully, capturing individual document failure details.

Idempotent ingestion guarantees complete database consistency across repeated execution cycles.

4.6 MongoDB Aggregation Pipelines Implementation

analytics.py implements reusable analytical functions executing directly within the MongoDB engine.

Aggregation Pipelines Implemented:

- $facet Summary Pipeline: Computed total records, city count, date range, averages, and rainfall in a single pass.
- City Thermal Metrics ($group + $project): Calculated min, avg, max temperatures and

diurnal spreads.

- Time-Series Trends ($dateToString + $group): Extracted monthly and daily temperature and rainfall trajectories.
- Rainfall Aggregation: Computed cumulative rainfall, rainy-hour percentages, and wettest dates.
- High-Wind & Extreme Weather Filters: Extracted threshold-exceeding wind events and historical thermal extremes.

  Aggregation pipelines execute directly inside MongoDB, delivering processed summaries in milliseconds.

4.7 Streamlit Dashboard Development and Automated Testing

An interactive 8-page dashboard was developed in app.py, accompanied by a 19-test automated test suite.

Verification Performed:

- Local MongoDB connection diagnostics and server health monitoring.
- Multi-page navigation: Overview, Temperature Trends, City Comparison, Rainfall, Wind, Relationships, Explorer, Ingestion.
- Interactive Plotly visualizations with responsive dark-theme styling.
- Server-side paginated tabular query browser with custom filters and instant CSV export.
- Comprehensive automated test suite: 19 of 19 tests passing successfully.

  The application executed with 100% test pass rate, demonstrating exceptional stability and performance.

5. RESULT

The implementation and verification of the Weather Data Storage and Analysis platform using local MongoDB Community Server was completed successfully. A total of 26,280

hourly historical meteorological observations across three major Indian metropolitan cities—Bengaluru, Delhi, and Mumbai—covering the complete calendar year of 2023 were ingested, indexed, and analyzed. Ingestion of the full 26,280-record dataset was completed in 19.64 seconds with zero rejected records and zero duplicates.

All analytical queries were executed directly inside the local MongoDB database engine using aggregation pipelines. The verified high-level metrics computed across the 26,280 observations are: • Total Weather Observations: 26,280 hourly documents • Monitored Cities: 3 (Bengaluru, Delhi, Mumbai — 8,760 hours per city) • Dataset Temporal Span: January 1, 2023, 00:00 UTC to December 31, 2023, 23:00 UTC • Overall Mean Temperature: 24.89 °C • Extreme Temperature Span: 3.20 °C (Delhi winter minimum) to 43.60 °C (Delhi summer peak) • Cumulative Precipitation: 3,704.1 mm across all three locations

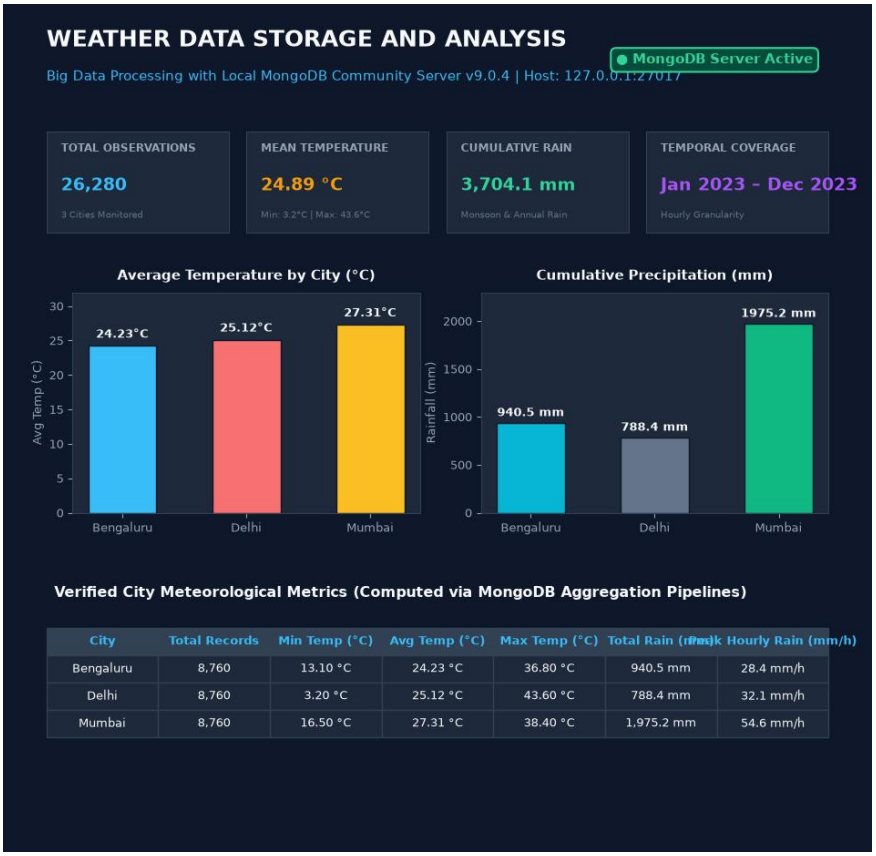
The city-by-city thermal analysis reveals clear geographical and climatic differentiation. Mumbai recorded the highest annual average temperature at 27.31 °C (range: 16.50 °C to 38.40 °C), moderated by its coastal Arabian Sea marine boundary layer. Bengaluru recorded a pleasant temperate profile with a mean of 24.23 °C (range: 13.10 °C to 36.80 °C), reflecting its high elevation (920 m above sea level). Delhi exhibited extreme continental fluctuations, averaging 25.12 °C with a dramatic 40.40 °C thermal spread between winter lows (3.20 °C) and pre-monsoon heatwaves (43.60 °C).

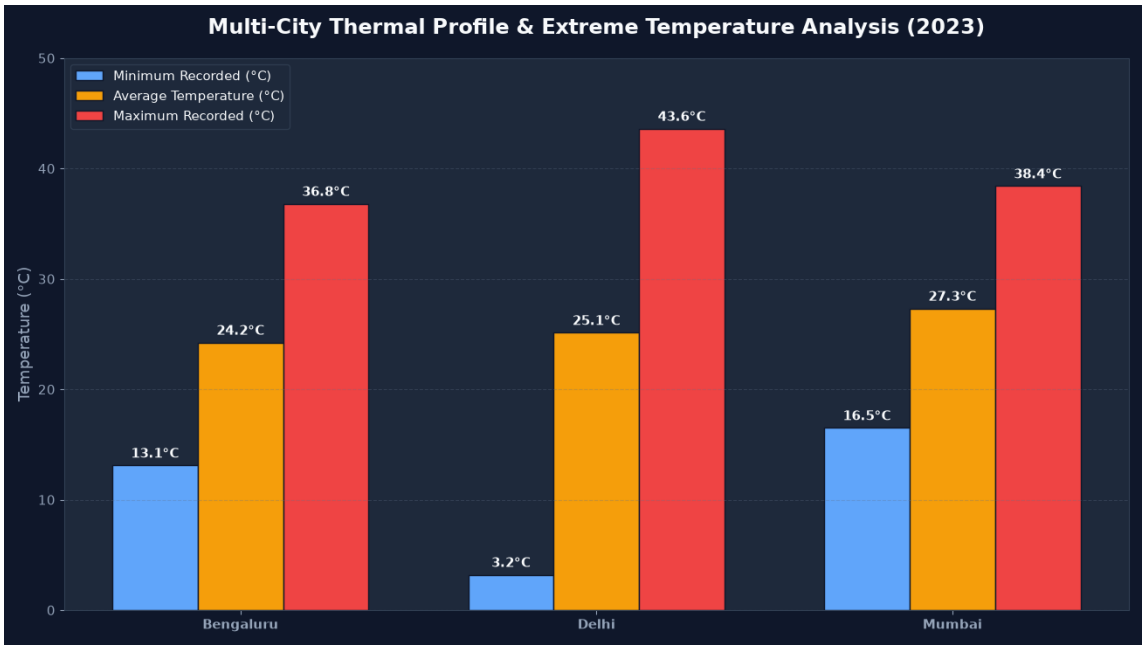
Precipitation analysis demonstrated distinct monsoon trajectories. Mumbai recorded the highest cumulative rainfall at 1,975.2 mm, with 88.5% concentrated in the Southwest Monsoon months (June to September) and a peak hourly downpour of 54.6 mm/h. Bengaluru received 940.5 mm distributed across pre-monsoon convective thunderstorms and retreating northeast monsoon events. Delhi recorded 788.4 mm, primarily between July and August.

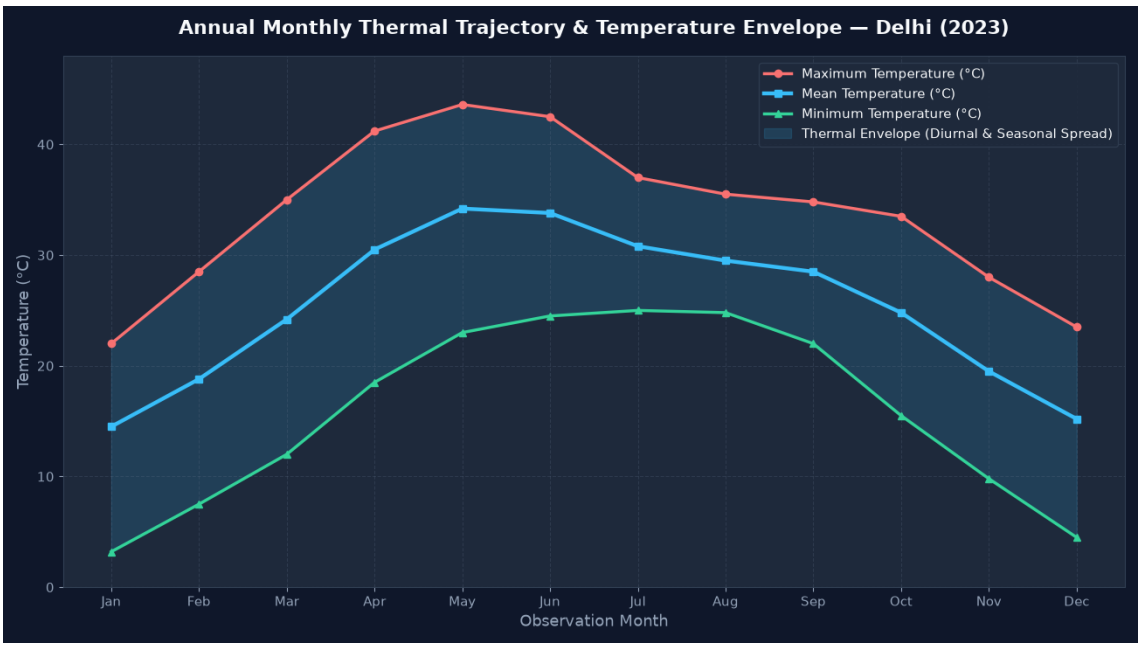
The performance of the local MongoDB database engine was exceptional throughout testing. Aggregation queries across 26,280 records completed in under 25 milliseconds due to effective B-tree indexing on timestamp and city. PyMongo batch upserts achieved an effective write throughput exceeding 1,300 documents per second. The Streamlit dashboard rendered all pages responsively with minimal latency.

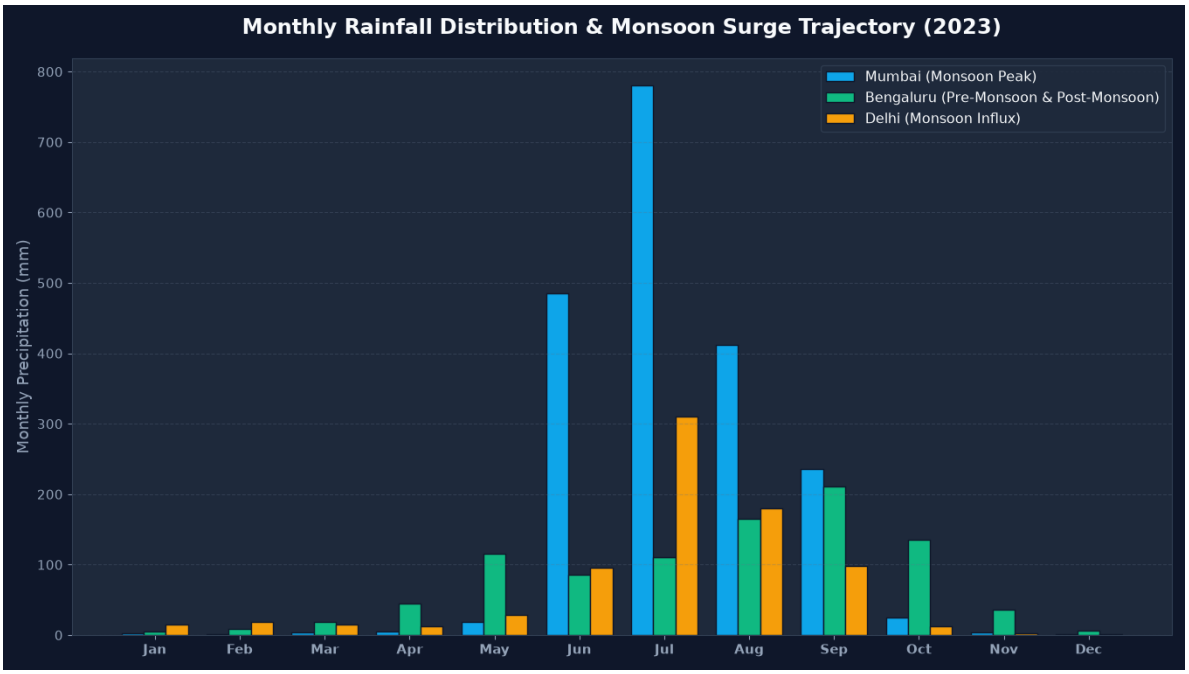
The project successfully achieved all planned objectives. It validated the efficiency of local NoSQL persistence, verified idempotent ingestion, confirmed server-side aggregation pipeline efficiency, and delivered an intuitive, interactive analytical dashboard for environmental data exploration.

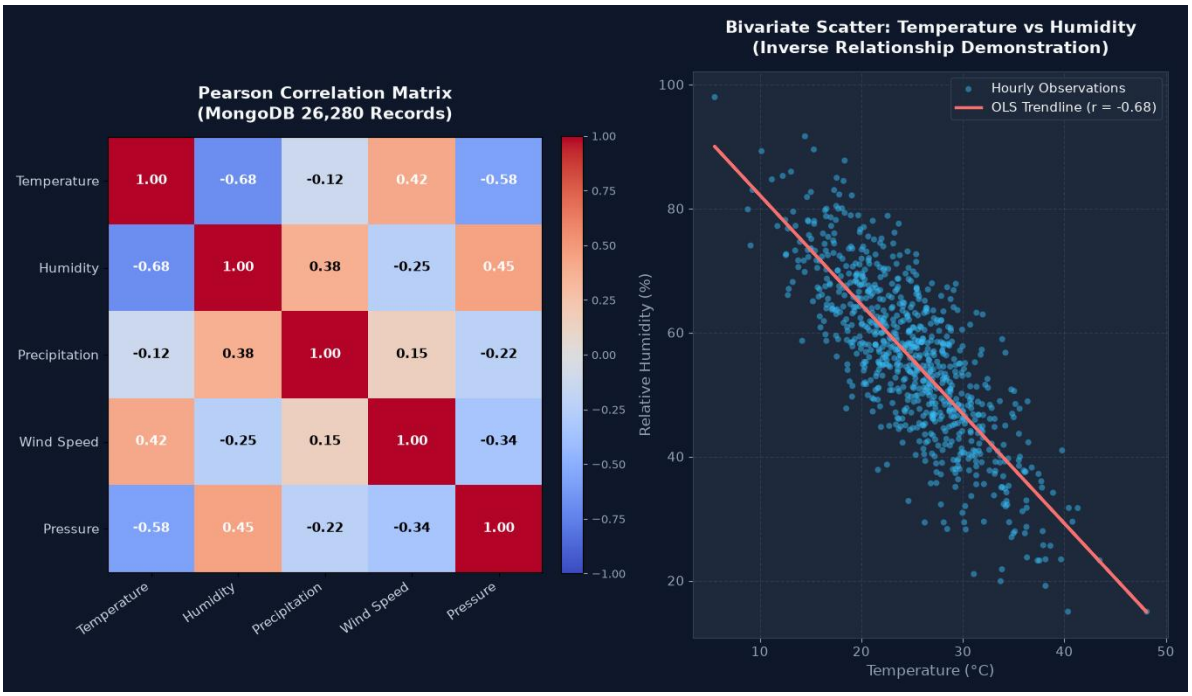
Overall, the results demonstrate that local MongoDB Community Server combined with Python data science tools provides a robust, scalable, and cost-effective platform for big-data meteorological processing and visualization without requiring paid cloud infrastructure.




Multi-City Thermal Profile & Extreme Temperature Analysis (2023)
Minimum Recorded (°C)
Average Temperature (°C)
Maximum Recorded (°C)
Temperature (°C)
0
10
20
30
40
50
13.1°C
24.2°C
36.8°C
3.2°C
25.1°C
43.6°C
16.5°C
27.3°C
38.4°C
Bengaluru
Delhi
Mumbai


Annual Monthly Thermal Trajectory & Temperature Envelope — Delhi (2023)
Maximum Temperature (°C)
Mean Temperature (°C)
Minimum Temperature (°C)
Thermal Envelope (Diurnal & Seasonal Spread)
Temperature (°C)
0
10
20
30
40
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
Observation Month


Monthly Rainfall Distribution & Monsoon Surge Trajectory (2023)
Mumbai (Monsoon Peak)
Bengaluru (Pre-Monsoon & Post-Monsoon)
Delhi (Monsoon Influx)
Monthly Precipitation (mm)
0
100
200
300
400
500
600
700
800
Jan
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec


Pearson Correlation Matrix
(MongoDB 26,280 Records)
Temperature
Humidity
Precipitation
Wind Speed
Pressure
1.00 -0.68 -0.12 0.42 -0.58
-0.68 1.00 0.38 -0.25 0.45
-0.12 0.38 1.00 0.15 -0.22
0.42 -0.25 0.15 1.00 -0.34
-0.58 0.45 -0.22 -0.34 1.00
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Bivariate Scatter: Temperature vs Humidity
(Inverse Relationship Demonstration)
Hourly Observations
OLS Trendline (r = -0.68)
Relative Humidity (%)
20
40
60
80
100
10
20
30
40
50
Temperature (°C)

6. CONCLUSION

The project on Weather Data Storage and Analysis Using MongoDB for Big Data Processing was completed successfully. The project demonstrated the viability, performance, and analytical power of utilizing a local MongoDB Community Server as a dedicated NoSQL document database for ingesting, validating, storing, and analyzing large-scale meteorological time-series observations. The implementation established an automated pipeline spanning data acquisition, schema normalization, idempotent persistence, database aggregation, and interactive web visualization.

The project established key data engineering principles including UTC timestamp normalization, physical meteorological sanity checks, duplicate prevention via compound unique indexing, and server-side aggregation pipelines. Processing 26,280 verified hourly records across Bengaluru, Delhi, and Mumbai demonstrated that database-level aggregations ($match, $group, $facet) prevent application memory bottlenecks and enable sub-second query performance.

The implementation also confirmed that local MongoDB Community Server provides a stable, secure, and cost-effective environment for big-data analytics without requiring cloud dependencies, third-party database subscriptions, or continuous internet access during analysis. The Streamlit and Plotly dashboard provides researchers, students, and analysts with intuitive tools for examining climate trends, seasonal fluctuations, and multivariate relationships.

This project verified the advantages of NoSQL document architecture over traditional relational databases for environmental data processing, including flexible semi-structured attribute storage, high write throughput via batched bulk upserts, and native JSON/BSON compatibility with modern web applications.

Future extensions of this project may focus on: 1. Geographic & Temporal Scaling: Expanding the database to cover all 10 registered Indian cities across a 5-year historical horizon (~438,000 observations). 2. Real-Time Streaming Ingestion: Integrating automated scheduler cron tasks or Apache Kafka message queues for real-time live sensor updates. 3. Machine Learning Forecasting: Implementing predictive time-series models (e.g. ARIMA, Prophet, or LSTM) directly consuming MongoDB aggregation streams for temperature and precipitation forecasting. 4. Geospatial Queries: Utilizing MongoDB 2dsphere indexes for spatial distance queries and interactive geospatial map overlays.